

2026 PORTFOLIO



Siho Kim

프로젝트

Linux r8152 USB NIC Driver 성능 분석

r8152 RX Path 최적화 : GRO, Buffer Size Tuning, and RPS

CONTENTS

1	개요	<u>4</u>
2	기술 배경지식	<u>6</u>
3	테스트	<u>12</u>
4	결과 해석	<u>18</u>

목표

r8152 USB NIC Driver에 GRO, RX buffer size, RPS 가 성능에 미치는 영향 분석

측정 metric 간의 trade-off 분석 및 여러 네트워크 상황에 알맞는 설정 값 확인

측정 metric

metric	측정 방법	설명
Throughput	iperf	동일 데이터 양 전송에 따른 Throughput 측정
CPU %soft	mpstat /proc /softirqs	전송 도중 softirq cpu 시간 측정
Packet count	/proc/net/dev	전송 전후 RX packet 차이
GRO merge ratio	계산	$1 - (\text{GRO on 패킷 수} / \text{GRO off 패킷 수})$
Latency (RTT)	ping	전송 테스트 수행 중 ping 으로 지연 측정
Memory usage	slabinfo	메모리 사용량 확인

개발 환경

- 커널 버전 6.19 rc5
- **USB NIC** RTL8153 1Gbps NIC (USB 3.2 GEN1)
- **개발 환경(가상 머신)** Virtime-Ng (Qemu) 사용 및 USB Nic Passthrough 로 부팅 후 GDB 디버깅
- **성능 테스트 환경(물리 머신)** host에 커널 설치 후 별도의 호스트에서 iperf 트래픽 전송

테스트 방법

- iperf 로 테스트 (고정된 시간 (30s), 패킷 크기 별 테스트(small / large))
- 총 3 가지 테스트 수행
 1. GRO on / off
 2. RX buffer size 튜닝
 3. RPS적용 / 미적용

R8152 RX 동작 설명

- PCIe NIC driver는 일반적으로 DMA descriptor ring을 직접 관리

반면 r8152와 같은 USB NIC은 URB(USB Request Block)를 통해 데이터를 전달받음

URB 에서 사용하는 버퍼는 DMA 가능한 메모리로 할당되며, USB host controller가 이를 통해 데이터를 전송

- driver는 최대 RTL8152_MAX_RX (10개)의 RX URB를 미리 USB core에 제출

NIC에서 데이터가 도착하면 USB host controller가 대기 중인 URB buffer 중 하나에 데이터를 채움

URB가 완료되면 completion handler가 호출되고, driver는 해당 URB를 다시 submit하여 RX 파이프라인을 유지

- NIC에서 여러 이더넷 프레임을 URB 버퍼 하나에 묶어서 전송. URB 하나를 수신하면 그 안에 이더넷 프레임이 여러 개 들어있을 수 있음

[rx_desc1 | frame1 | rx_desc2 | frame2 | rx_desc3 | frame3 | ...]

rx_desc에는 해당 프레임의 길이, 상태 등 메타데이터 저장

이 방식은 USB NIC에서 흔히 사용하는RX aggregation 방식

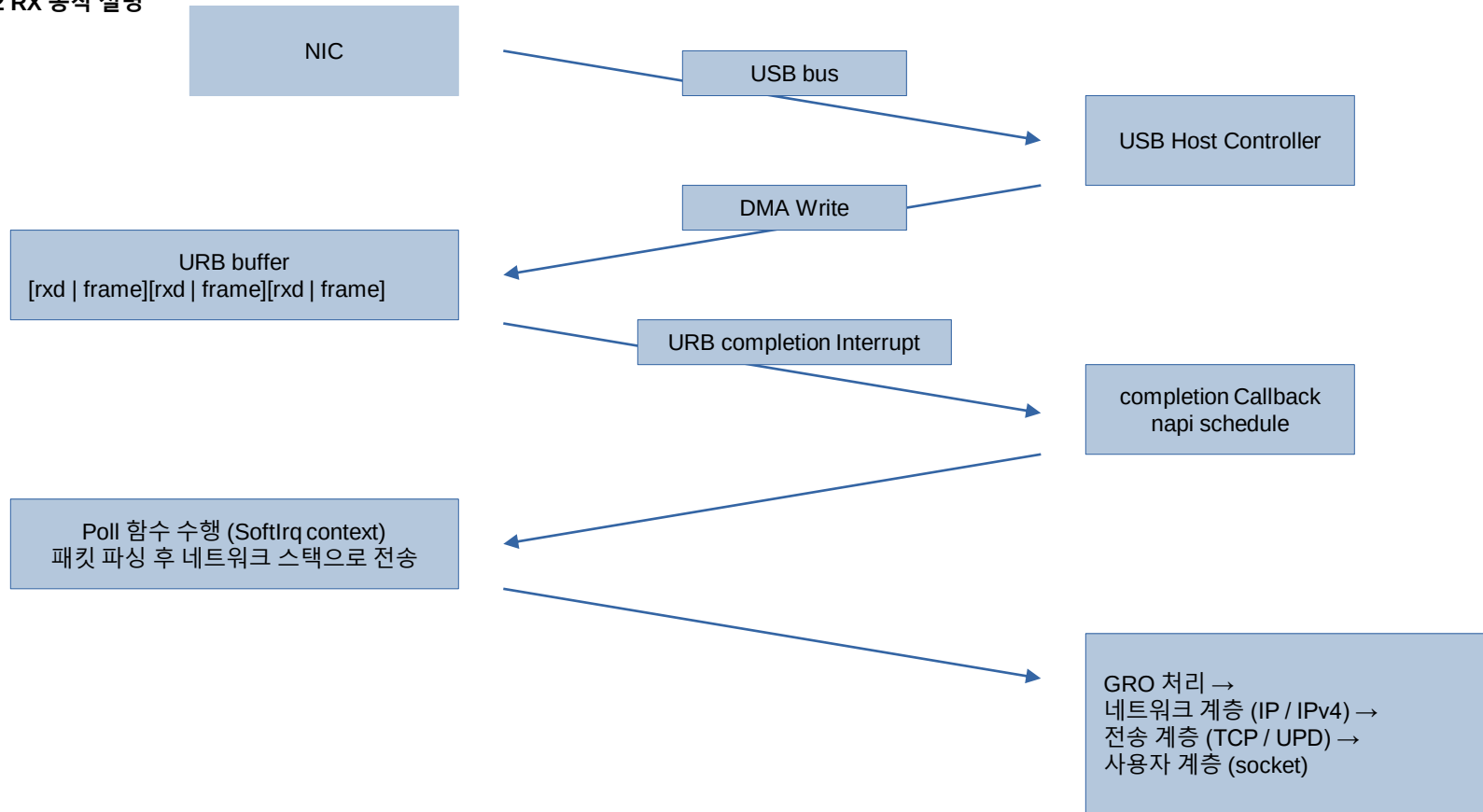
여러 이더넷 프레임을 하나의 URB buffer에 모아 USB transaction overhead를 줄인다

- URB 전송 완료 인터럽트에서 NAPI 를 schedule 하고 NAPI 의 polling 함수가 NET_RX_SOFTIRQ 서비스에서 URB 를

통해 받아온 데이터를 순회 하면서 파싱하고 커널 네트워크 스택으로 올림

(rx_desc 읽어서 frame 크기 파악 → 해당 크기 만큼 frame 추출해서 커널 네트워크 스택으로 올림 → 다음 descriptor 이동 → 버퍼 끝까지 반

R8152 RX 동작 설명



GRO 기본 동작

- GRO는 GRO 진입 → 패킷의 네트워크 계층 검사 → 패킷의 전송 계층 검사 → GRO 처리 순서로 진행

- 네트워크 계층 검사의 경우, ipv4는 다음과 같은 조건을 검사

옵션 없는 기본 헤더, IP fragment 패킷은 불가, IP 헤더 체크섬 검사, 같은 flow (protocol, source ip, dest ip)

- 전송 계층 검사의 경우, tcp 는 다음과 같은 조건을 검사

TCP flag 조건 검사, ACK/SEQ 검사, TCP 옵션 검사, TCP 헤더 체크섬 검사, 같은 flow (source port, dest port)

- 병합된 skb는 하나의 덩어리로 네트워크 스택에 올라감

여러 개를 각각 올리는 게 아니라 한 번에 올리니까 네트워크 스택 호출 횟수가 줄어듦

네트워크 스택은 프로토콜 스택 처리, 소켓 lookup, skb 관리 등 고정 비용 발생

GRO 병합은 GRO 관련 처리만 하면 되서 훨씬 저렴

그래서 N번 호출할 비용 대신 병합 비용 1번 + 호출 비용 1번으로 처리할 수 있어서 CPU 사용률이 낮아지는 효과를 얻을 수 있음

RX buf size 기본 개념

- rx_buf_sz는 URB 하나당 DMA 버퍼 크기를 결정

- 버퍼가 클수록 USB bulk transfer 한 번에 더 많은 이더넷 프레임이 전달될 수 있음

그러면 NAPI poll 한 사이클에서 GRO가 처리할 수 있는 프레임 수가 늘어나고 아래와 같은 효과가 나타남

- URB 하나에 더 많은 프레임
- NAPI poll 한 번에 더 많은 프레임
- GRO 병합 가능성 증가
- TCP 스택 호출 횟수 감소
- CPU 사용률 감소

- Trade-off로는, 버퍼가 클수록 한 번의 USB 전송에 더 많은 프레임이 묶여 전달되므로 패킷 전달 latency가 증가할 수 있음

고트래픽 환경에서는 프레임이 버퍼에 쌓여서 USB 전송이 완료되기까지 더 오래 기다려야 하므로 지연이 증가

또한 증가한 버퍼 크기만큼 메모리 사용량이 증가

RPS 기본 동작

- URB 콜백(irq context)에서 softirq 스케줄된 시점부터 네트워크 스택까지 모든 수신 처리가 softirq 컨텍스트에서 실행

그런데 USB 인터럽트를 처리하는 CPU가 IRQ affinity로 특정 CPU에 주로 바인딩 되어서, 이 모든 처리가 특정 CPU에서만 실행됨

그 말은 softirq 처리 전부, 즉 NAPI poll, GRO, TCP 스택까지 모두 한개의 코어에서 실행된다는 것이다

나머지 CPU는 네트워크 수신 처리에 거의 관여하지 않고 있음

단일 CPU에 처리가 집중되는 이 구조적 한계 때문에 RPS 같은 기술이 필요

RPS를 통해 네트워크 스택 처리를 다른 CPU로 분산시키는 게 목적

- CPU 로 분산시킬 때, flow (src ip, dst ip, src port, dst port, protocol) 기반 hash 를 계산하고,

해당 hash 값과 '어떤 CPU들을 RPS 용으로 설정하겠다' 명시한 CPU mask 값을 사용해 특정 CPU 번호를 구하고,

해당 CPU의 백로그 큐에 enqueue 하고 NET_RX_SOFTIRQ 에서 처리

RX 전체 흐름

Stage	Context	Key Function	Description
URB complete	HW interrupt	read_bulk_callback	USB URB 완료, rx_done list 추가, napi_schedule
NAPI schedule	HW interrupt	napi_schedule	NET_RX_SOFTIRQ 예약
NAPI poll	softirq	r8152_poll	최대 budget 만큼 RX 처리 시작
Frame parsing	softirq	rx_bottom	URB buffer에서 Ethernet frame 추출 → skb 생성
GRO entry	softirq	napi_gro_receive	GRO engine 진입
GRO merge	softirq	inet_gro_receive → tcp_gro_receive	flow 검사 후 TCP segment merge
RPS decision	softirq	get_rps_cpu	flow hash 기반 CPU 결정
RPS enqueue	softirq	enqueue_to_backlog	선택된 CPU backlog queue로 전달
backlog poll	softirq	process_backlog	backlog queue에서 skb dequeue
L2 receive	softirq	__netif_receive_skb	Ethernet protocol dispatch
L3 processing	softirq	ip_rcv	IPv4 header 검사 및 routing
L4 processing	softirq	tcp_v4_rcv → tcp_rcv_established	TCP 상태 처리 및 socket lookup
socket deliver	softirq	tcp_data_queue	socket에 packet enqueue

GRO on / off

- 테스트 진행하면서 perf 로 네트워크 스택 진입 호출 횟수 추적

```
ethtool -K <interface> gro <on/off>
```

```
iperf -s
```

```
iperf -c <server ip> -t 30 -P 4
```

```
perf stat -e net:netif_receive_skb
```

- GRO on 의 경우 GRO 처리 관련 metric 도 추적

```
perf stat -e net:napi_gro_receive_entry
```

```
perf record -e net:napi_gro_receive_exit
```

```
perf script
```

GRO on / off

- GRO 활성화 시 네트워크 스택 진입 횟수(`netif_receive_skb`)가 크게 감소하는 것을 확인

GRO는 여러 패킷을 하나의 큰 `skb`로 병합하기 때문에 네트워크 스택 진입 횟수는 줄어들지만

한 번에 더 많은 데이터를 처리하게 되어 전체 처리량이 증가

- GRO 비활성화 시, 패킷 병합이 이루어지지 않아 네트워크 스택 처리 효율이 감소하고 전체 처리량이 감소

CPU가 병목이 아닌 환경에서는 GRO 비활성화로 처리량이 감소하면서 전체 처리되는 패킷 수가 줄어들어 CPU softirq 사용률도

함께 감소하는 현상이 관찰

Metric	GRO ON	GRO OFF	Delta
Throughput	약 600mbps	약 500mbps	20% 증가
net:netif_receive_skb events	220,000	1,160,000	81% 감소
CPU %soft	~25%	~15%	10% 증가

Rx buf size 튜닝

- r8152 드라이버의 rx_buf_sz 값을 수정하여 모듈을 다시 빌드하고 교체하여 테스트 수행

기본 rx_buf_sz 는 32KB이며, 이는 8개의 page(order 3) 를 사용

Linux page allocator는 2의 거듭제곱 단위(order) 로 메모리를 할당하므로 다음 page 단위는 order 4 (16 pages = 64KB)

따라서 rx_buf_sz 튜닝은 page order 기준으로 테스트 수행

order 3 → 32KB, order 4 → 64KB

- 실행 명령어

iperf -s

iperf -c <server ip> -t 30 -P 4

ping <server ip> (iperf 테스트 수행 도중)

Rx buf size 튜닝

- 현재 드라이버는 10개의 RX URB buffer 를 사용하므로 총 RX buffer 메모리 사용량은 다음과 같이 증가

$$32\text{KB} \times 10 = 320\text{KB} / 64\text{KB} \times 10 = 640\text{KB}$$

MTU 1500 기준 Ethernet frame $\approx$ 1500 byte

32KB buffer $\rightarrow$ 약 21개의 frame 수용 / 64KB buffer $\rightarrow$ 약 42개의 frame 수용

즉 rx_buf_sz 증가 시 한 번의 URB에서 더 많은 frame을 처리할 수 있음

Buffer 크기 증가 $\rightarrow$ URB 당 처리 frame 증가 $\rightarrow$ GRO 병합 가능성 증가 $\rightarrow$ CPU 사용률 감소 가능

- 단일 비교 실험에서는 rx_buf_sz를 64KB로 확장했을 때, 32KB 대비 더 높은 throughput이 관찰

반면에 iperf 테스트 수행 도중 ping 명령어로 지연 시간 확인 결과는 조금 느려진 것을 확인

다만 테스트 반복 횟수가 제한적이므로 절대적인 수치 비교보다는 성능 개선 경향을 확인한 결과로 해석

RX buf sz	throughput	Latency RTT	Memory
32KB	약 500mbps	47ms	320KB
64KB	약 600mbps	56ms	640KB

RPS 적용 / 미적용

- 지금 나의 테스트 환경은 CPU 가 병목이 되고 있지는 않지만, NIC 성능이 더 높거나 CPU 성능이 낮은 환경에서는 패킷 처리가 특정 CPU에 집중되어 CPU 가 병목이 될 수 있음

현재 테스트 환경에서는 USB NIC 인터럽트가 CPU6에 할당되어 있으며 (IRQ affinity), 기본적으로 패킷 처리가 CPU6에서 시작 코어 수는 22개 이다.

- RPS 적용은 0~3 코어에 설정

```
echo f > /sys/class/net/<interface>/queues/rx-0/rps_cpus
```

- iperf 로 테스트 수행하는 동안 mpstat 명령어로 cpu 사용량 확인

```
iperf -c <server ip> -t 30 -P 4
```

```
mpstat -P ALL 2
```

```
cat /proc/net/softnet_stat
```


RPS 적용 / 미적용

- 현재 테스트 환경에서는 CPU가 병목이 아니기 때문에 RPS 적용 여부에 따른 처리량(Throughput) 차이는 발생하지 않음
- 현재 테스트에서는 CPU 부하가 높지 않아 RPS 적용에 따른 성능 향상은 체감되지 않지만,
패킷 처리가 여러 CPU로 정상적으로 분산되는 것은 확인
트래픽이 많은 환경에서는 RPS의 효과가 있을 것으로 예상됨

Metric	RPS OFF	RPS ON	Description
Throughput	약 600mbps	약 600mbps	차이 없음 cpu 병목 아님
Cpu 6 %soft	~27%	~25%	일부 패킷 처리가 다른 CPU로 분산됨
CPU 0~3 %soft	~0%	각 ~5%	RPS에 의해 패킷 처리 수행

핵심 결론

GRO는 여러 패킷을 하나의 큰 skb로 병합하여 네트워크 스택 진입 횟수를 감소시킴

실험 결과 netif_receive_skb 호출 횟수가 약 81% 감소하였고, 그에 따라 softirq 처리 비용 감소로 전체 처리량이 약 20% 증가

즉 GRO는 패킷 처리 효율을 높여 CPU 사용을 효율적으로 만들고 처리량 향상에 직접적인 영향을 줌

rx_buf_sz는 버퍼를 크게 하면 GRO 병합 가능성이 높아져서 트래픽이 많은 환경에서 처리량이 향상

하지만 지연 시간과 메모리 사용량이 소폭 증가

RPS는 수신 패킷의 softirq 처리를 여러 CPU로 분산시키는 기능

실험 환경에서는 NIC 인터럽트가 CPU6에 집중되어 있었으며, RPS 적용 시 패킷 처리 softirq가 다른 CPU로 분산되는 것을 확인

다만 CPU가 병목이 아닌 상황에서는 처리량 자체에는 큰 변화가 없었음

USB NIC는 일반적으로 하나의 Bulk IN endpoint를 통해 수신 데이터가 전달되며, 이에 따라 단일 RX 처리 경로(NAPI)가 사용되는 경우가 많다

따라서 패킷 처리가 특정 CPU에 집중되는 경향이 있음

이러한 구조에서는 하드웨어 기반 멀티큐를 활용하기 어렵기 때문에, RPS와 같은 소프트웨어 기반 부하 분산 기법을 통해

여러 CPU를 활용해서 CPU 부하를 분산시킬 수 있음

따라서 USB NIC 환경에서는 하드웨어 멀티큐를 활용하기 어렵기 때문에 GRO를 통한 처리 효율 향상과 RPS를 통한 CPU 부하 분산이

성능 개선에 도움을 줄 수 있음

Siho Kim

Thank You